Лабораторная работа №2
Тема: Сегментация и классификация фишек Тантрикс на изображении
Дисциплина: Обработка и распознавание изображений

Выполнил: студент группы 317
Глазунов Матвей

Преподаватель: Местецкий Л. М.

26 апреля 2026 г.

1 Постановка задачи

Целью работы является разработка и программная реализация алгоритма для автоматического распознавания фишек игрового набора «Тантрикс». В данной работе я хотел найти простой, легкий и надежный алгоритм классификации фишек, способный работать из коробки без дополнительных параметров, эвристик или машинного обучения.

Программа должна обеспечивать:

- Ввод и отображение изображений.
- Сегментацию объектов (фишек) на основе точечных и пространственных преобразований.
- Генерацию признаковых описаний фишек.
- Классификацию отдельных фишек (определение их номера).

2 Описание данных

Входные данные представляют собой растровые изображения формата BMP. На изображениях представлены черные шестиугольные фишки с нанесенными на них сегментами линий трех цветов: синего, красного и желтого. Сложность данных варьируется от одиночных объектов (`Single_*.bmp`) до плотных групп соприкасающихся фишек (`Group_*.bmp`) и замкнутых маршрутов (`Path_*.bmp`).

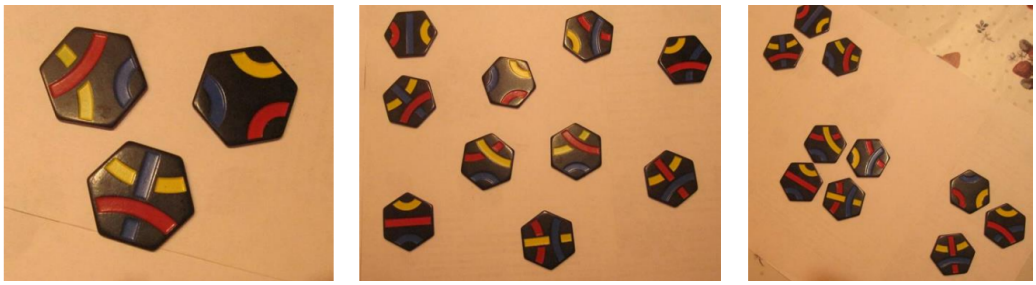

Рис. 1: Пример входных данных

3 Описание метода решения

3.1 Сегментация

Фон на всех изображениях простой и монотонный, поэтому алгоритм сегментации тоже простой. Поэтапно его можно описать следующим образом:

1. **вход:** Изображение в BGR формате.
2. **Предобработка:** Применение медианного фильтра (`medianBlur`) для подавления импульсного шума.
3. **Сегментация:**
   - использование детектора границ Кэнни;
   - применение морфологической операции закрытия (дилатация + эрозия) с ядром 3, чтобы лучше связать границы фишек;
   - поиск внешних контуров и их выпуклых оболочек (на основе `cv2.findContours`), так как все шестиугольники выпуклые:
   - применение операции эрозии, чтобы разделить соприкасающиеся компоненты и отделить их маски от фона;
   - использование метода связных компонент (основан на стандартном алгоритме система непересекающихся множеств, время работы линейное относительно числа пикселей в маске).
4. **Результат:** Отдельная маска на каждую фишку на изображении.

3.2 Классификация

Идея следующая: всего есть 10 классов, нам известно, как они выглядят — если мы научимся на основе пары (изображение, маска) получать вектор признаков, то очередную маску мы можем сравнить с эталонными примерами (достаточно одного на каждый класс); у кого из эталонов векторы признаков больше всего похожи, к такому классу принадлежит наша очередная маска. То есть алгоритм следующий:

- `get_features(image, mask)` — функция, на выходе которой мы получаем вектор признаков.
- {`image1`, ..., `image10`} — картинки, на каждой из которых содержится одна фигура, то есть наши эталонные примеры, номер картинки соответствует ее классу (сделать крайне просто и быстро).
- на вход мы получили очередную маску `mask` на изображении `image`, нужно понять, к какому классу она относится при помощи сравнения с эталонными изображениями. Мы будем относить к тому классу, у которого мера косинусного сходства ($\texttt{cos_sim}(a, b) = \frac{\langle a,b\rangle}{\|a\|\|b\|}$) будет наибольшая, то есть

  ```
  cos_sim(get_features(image, mask), get_features(image[i], get_mask(image[i])))
  ```

`get_mask` был описан выше. Как именно выглядит `get_features(image, mask)`? Данная функция должна быть инвариантна к **сдвигу, масштабу, повороту и освещенности** каждой маски по-отдельности.

Чтобы быть инвариантным к освещенности предлагается перейти от формата BGR к LAB, при этом мы будем работать только с канали A и B, без канала L.

Но как же нам из маски получить вектор признаков? Есть множество способов, в моей работе я придумал использовать следующий алгоритм, пусть для начала мы работаем только с одним каналом A:

1. найти центр массы маски;

2. найти радиус вписанной окружности маски по этому центру $radius$ (в данном случае мы уже знаем, что маска шестиугольная, поэтому радиус можно посчитать просто как $radius = \sqrt{\frac{area}{2\sqrt{3}}}$);

3. всего на канал у нас будет n признаков, нам нужно на выходе вернуть вектор размера n, назовем его `vec_a`. Тогда `vec_a`[i] = среднее значение канала A внутри кольца с внешним радиусом $radius * \frac{i}{n}$, внутренним радиусом $radius * \frac{i-1}{n}$, и цетром в центре массы маски, $i = 1...n$. Данная операция проиллюстрированна на изображении 2.

4. полученные векторы для каналов A и B, под названиями `vec_a` и `vec_b` соответственно, мы объединяем в один вектор `vec` и нормируем его, чтобы быть инвариантными к насыщенности цветов.

Рис. 2: Кольца, по которым мы считаем средние значения в канале (n=10)

Чем обусловлен выбор такого алгоритма? Во-первых, он крайне легкций (число операций линейно относительно числа пикселей на изображении) и очень простой, что обеспечивает его гибкость и надежность, нам не нужно надеяться, что мы выделим нужные контуры в маске, что не будет бликов, теней и тому подобное. Во-вторых, благодаря тому, что мы используем именно кольца, алгоритм инвариантен к повороту изображения и маски, мы всегда будем получать один и тот же вектор признаков, как бы мы не вращали объект (фишку); благодаря тому, что мы используем относительные радиусы в кольцах (относительно радиуса вписанной окружности) и средние значения, алгоритм инвариантен к масштабу, нам не нужно искусственно изменять размерность изображения или двигать камеру; благодаря использованию цетнра маски мы инвариантны к сдвигу, не нужно обрезать изображение; благодаря использованию LAB-представления мы инвариантны к освещенности. То есть все необходимые критерии для `get_features` выполнены.

4 Описание программной реализации

Программный комплекс реализован на языке Python с использованием библиотеки компьютерного зрения `OpenCV` и библиотеки `NumPy`. Графический интерфейс построен на базе фреймворка `Streamlit`. Код приложен к отчету.

Основные функции:

- `get_mask(image)` — выполняет фильтрацию, поиск границ и морфологическую обработку (секция "Сегментация").
- `load_templates()` — инициализирует базу данных эталонных признаков.
- `get_features(image, mask)` — вычисляет вектор признаков для маски на изображении (секция "Классификация").

```python
def get_features(img : np.ndarray, mask : np.ndarray):
    for i in range(10):
        r = int(radius * (i + 1) / 10)
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (r * 2 + 1, r * 2 + 1))
        center_crop = lab[center[0] - r : center[0] + r + 1, center[1] - r : center[1] + r + 1]
        a, b = (center_crop * kernel[:, :, None]).sum(axis = (0, 1))

        vec_a[i] = a
        vec_b[i] = b

    for i in reversed(range(10)):
        r1 = int(radius * (i + 0) / 10)
        r2 = int(radius * (i + 1) / 10)
        area = np.pi * ((r2 - r1) ** 2)

        if i == 0:
            vec_a[i] = vec_a[i] / area
            vec_b[i] = vec_b[i] / area
        else:
            vec_a[i] = (vec_a[i] - vec_a[i - 1]) / area
            vec_b[i] = (vec_b[i] - vec_b[i - 1]) / area

    features = np.concatenate((vec_a, vec_b))
    features /= np.linalg.norm(features) + 1e-10

    return features
```

Листинг 1: Реализация расчета радиальных признаков

5 Эксперименты

5.1 Сегментация

На рисунке 3 представлены этапы обработки входного изображения: выделение границ, построение выпуклых оболочек и финальная маска после разделения объектов. На выданных тренировочных данных все объекты были успешно сегментированы, то есть полнота алгоритма составляет 100%.

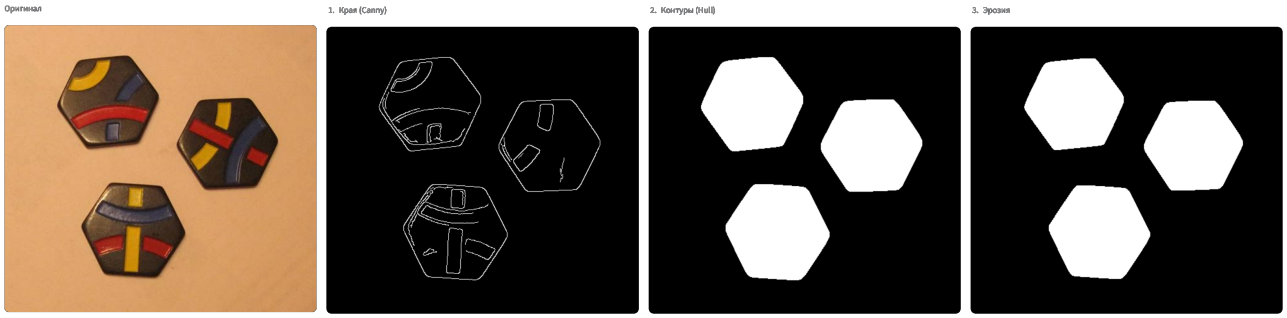

Рис. 3: Этапы обработки изображения: Original, Canny, Hull, Erode

5.2 Классификация

Ниже представлен пример работы классификатора. Пришлось глазками проверить, сколько объектов было верно классифицированно. Итого было верно классифицированно 90.4% объектов, то есть это наша итоговая точность.

В сравнении в более сложными методами результат не самый высокий, но, с учетом простоты алгоритма, результат очень хороший. Учитывая надежность и гибкость алгоритма, его легко можно использовать как бейзлайн для более сложных алгоритмов.

Рис. 4: Итоговый результат классификации фишек

5.3 Надежность

На графике 5 предствлена матрица сходства между векторами признаков шаблонов классов, метрика сходства все та же — `cos_sim`. На этой матрице мы видим, что векторы признаков заметно отличаются друг от друга для почти каждого класса (мера сходства между разными классами не больше 0.9 для всех пар разных шаблонов, кроме трех). Это говорит о том, что даже такой алгоритм способен надежно отличать фишки друг от друга. На этом же графике мы видим, что некоторые шаблоны оказались близки (3 пары из 45). Вероятно, именно эти близкие классы повлияли на этоговую точность. Возможно, метрика `cos_sim` не является наилучшей в таком алгоритме, и можно улучшить качество, при помощи другой, более подходящей метрики, которая будет учитывать специфику наших признаков, но в данной работе таких исследований не будет.

Рис. 5: Матрица сходства

6 Выводы

В ходе выполнения лабораторной работы был разработан и реализован алгоритм сегментации и распознавания игровых элементов «Тантрикс».

Ключевыми результатами работы являются:

1. **Эффективная сегментация:** Комбинация детектора Кэнни, выпуклых оболочек и морфологической эрозии позволила достичь 100% полноты выделения объектов.

2. **Инвариантный дескриптор:** Реализованный метод радиальных цветовых профилей в пространстве LAB обеспечил устойчивость классификации к повороту, сдвигу, изменению масштаба и условиям освещенности без использования тяжелых вычислительных методов или обучения нейронных сетей.

3. **Анализ точности:** Достигнутая точность классификации (90.4%) подтверждает адекватность выбранной модели признаков. Матрица сходства выявила «узкие места» — пары классов со схожей цветовой топологией, что дает направление для

дальнейшего улучшения алгоритма (например, за счет анализа графика, который обазуют признаки).

Разработанная система является легковесной, работает в режиме реального времени и может служить надежным базовым решением (baseline) для более сложных задач компьютерного зрения.

Литература

1. Гонсалес Р., Вудс Р. Цифровая обработка изображений. М., Техносфера, 2006.